LEC 01 Shift-Left & SDLC | LEC 02 Secure Coding & Habits | LEC 03 12 Principles & STRIDE | LEC 04 Infra, Cloud & Network | LEC 05 Docker, K8s & RBAC | LEC 06 CI/CD, SAST/DAST/SCA | LEC 07 SIEM, Logs & IR 6-Steps | LEC 08 GRC, GDPR & HIPAA | LEC 10 Serverless & DoW

FINAL EXAM ROUTER: Q1 MCQs: All Lectures (25 Marks) | Q2 Structured: STRIDE (L03) · CI/CD Gates (L06) · Docker Arch (L05) (25M) | Q3 Bank Essay: Shift-Left (L01) · Secure Coding (L02) · STRIDE (L03) · Infra (L04) (25M) | Q4 Health Essay: CI/CD (L06) · Docker/K8s (L05) · GRC/GDPR (L08) · CSM/IR (L07) (25M)

1. DevSecOps & Shift-Left SDLC

LEC 01 | Q1, Q3

DevSecOps: Integrating security across every phase of SDLC. Security is a **shared responsibility** of Dev, Sec, and Ops (not an isolated final gate).

Shift Left: Moving security activities to the *earliest stages* (Requirements & Design) instead of waiting for pre-release testing.

RELATIVE COST OF FIXING DEFECTS (COST MULTIPLIER)

SDLC Phase	Activity / Security Control	Cost Multiplier
1. Requirements	Security & abuse cases, compliance goals	**1×** (Cheapest)
2. Design	**Threat modeling (STRIDE)**, secure arch	**5×**
3. Coding	Secure coding, pre-commit hooks, IDE lint	**10×**
4. Build / Test	SAST, SCA, secrets scan, DAST, gates	**15×**
5. Production	Breach, emergency hotfix, legal fines	**100×** (Disastrous)

TRADITIONAL SDLC VS DEVOPS VS DEVSECOPS

Aspect	Traditional	DevOps	DevSecOps
Speed	Slow, quarterly	Rapid, continuous	Rapid & secure
Security	Perimeter gate at end	Often bypassed	Built-in automated
Owner	Isolated Sec team	Dev & Ops	**Shared by all**

Cost & Security Benefits: Early detection lowers rework cost; automated pipeline checks provide fast feedback, prevent human error, and stop bad code from reaching prod.

2. Five Core Secure Coding Practices

LEC 02 | Q1, Q3(b)

1. Parameterized Queries (Prepared Statements): Defeats SQLi (OWASP A03). Database treats user input strictly as *literal data*, never executable SQL command.

```
// VULNERABLE: "SELECT * FROM users WHERE name = '" + name + "'"
// SECURE: cursor.execute("SELECT * FROM users WHERE name = %s", (name,))
```

2. Allow-list Input Validation: Define allowed chars, length, type, regex (e.g. `/^[a-zA-Z0-9_]{3,20}$/`). **Fails closed** against unknown attacks. Reject invalid input (never "sanitize & use"). Canonicalize/decode *before* validation.

3. Context-Aware Output Encoding: Defeats XSS (A03). Converts special characters to safe entities (`<` → `<`, `>` → `>`, `"` → `"`) at browser display time.

4. Memory-Hard Password Hashing: Never use MD5/SHA-1/SHA-256. Use **Argon2id**, **bcrypt** (work factor ≥ 12), or **PBKDF2** with unique salt per user to defeat GPU brute-force.

5. Complete Mediation & Server-Side AuthZ: Verify authentication & role on *every request*. Never trust client-side flags.

```
if (user.role != "ADMIN") return response.status(403).send("Forbidden");
```

DB Least Privilege: App DB user gets only `SELECT, INSERT, UPDATE` (never `DROP`, `ALTER`, or `GRANT`).

AUTHN VS AUTHZ & SECURE LOGGING

- **AuthN:** Validating identity (MFA, password, WebAuthn).
- **AuthZ:** Enforcing permissions (RBAC, ABAC, least privilege).
- **Log MUST include:** UTC timestamp (NTP), user ID, IP, event, result.
- **Log NEVER contains:** Passwords, API tokens, PAN (credit card), PII.

3. Complete 12 Secure Design Principles

LEC 03 | Q1

Principle	Exam Definition & Implementation
Defense in Depth	Layered controls; if one fails, next blocks attack (WAF + mTLS + DB encryption).
Fail Securely	On error/exception, lock system state and revoke access (fail closed, not open).
Least Privilege	Grant minimum permissions required for task, for the minimum duration.
Fail-Safe Defaults	Default state is deny/no access until explicitly granted (e.g. firewall deny-all).
Secure Defaults	Out-of-box config has highest security enabled (require password change, TLS).
Minimize Surface	Disable unused ports, remove dead endpoints, prune dependencies.
Separation of Duties	Require ≥ 2 distinct parties for sensitive actions (e.g. dev cannot merge).
Complete Mediation	Validate authorization on **every access attempt**; never cache past checks.
Economy of Mechanism	Keep design as simple and small as possible; simplicity prevents flaws.
Open Design	Security must not depend on secret algorithms; assume attacker knows design.
Least Common Mech.	Minimize shared state/resources between users to prevent covert channels.
Psych. Acceptability	Security must be easy and seamless so users do not bypass it.

4. Threat Modeling, STRIDE & DREAD

LEC 03 | Q1, Q2(i), Q3(c)

Threat Modeling: Structured process in **Design Phase** to identify, analyze, and mitigate threats before code is written.

4 Core Questions: 1. What are we building? 2. What can go wrong? 3. What will we do about it? 4. Did we do a good job?

Why in Design Phase? Shift Left: architectural flaws are caught before implementation; saves rework costs; establishes test cases & security requirements.

Assets: High-value targets (Student/Patient data, credentials, exam questions, marks/results, transactions, audit logs, API keys).

Trust Boundary: Where data passes between different privilege levels (e.g. Client Browser → API Gateway → DB Server).

STRIDE THREAT CLASSIFICATION & CONTROLS

Threat	Violates	Exam Scenario Example & Control
S – Spoofing	Authentication	Attacker logs in with stolen user password. → *MFA, strong password policy, account lockout*
T – Tampering	Integrity	Student/attacker alters marks or transaction amounts. → *Server-side validation, HMAC, digital signatures*
R – Repudiation	Accountability	Lecturer/user denies altering marks/sending money. → *Tamper-proof audit logs + UTC time*
I – Info Disclosure	Confidentiality	SQLi exposes student marks or patient records. → *Parameterized queries, AES-256, TLS 1.3*
D – Denial of Svc	**Availability**	Flooding results portal with requests on release day. → *Rate limiting, WAF, auto-scaling*
E – Elev. of Priv.	Authorization	Student calls `/admin/users` directly. → *RBAC, server-side role check, least privilege*

DREAD RISK SCORING & RISK TREATMENTS

DREAD Formula: Risk Score = (Damage + Reproducibility + Exploitability + Affected Users + Discoverability) ÷ 5 (Scale 1–10). Score ≥ 7.0 = High Priority.

4 Treatments: Mitigate (controls), **Avoid** (drop feature), **Transfer** (cyber insurance/SaaS SLA), **Accept** (signed executive approval).

5. Infrastructure & Cloud Security

LEC 04 | Q1, Q3(d)

Cloud Shared Responsibility: CSP manages **Security OF the Cloud** (physical DC, hardware, hypervisors). Customer manages **Security IN the Cloud** (data, IAM, guest OS, network config, apps).

CLOUD SERVICE MODELS RESPONSIBILITY

Model	CSP Manages	Customer Responsible
IaaS (EC2)	Hardware, facilities, hypervisor	**Guest OS patching**, apps, IAM, data
PaaS (RDS)	Hardware, OS, DB engine	App code, DB schemas, IAM, data
SaaS (O365)	Entire infrastructure & application	User access, data classification

AWS VS AZURE SECURITY SERVICES MAPPING

Capability	AWS Service	Azure Service
Identity & Access	AWS IAM / SSO	Microsoft Entra ID
API Activity Audit	CloudTrail	Azure Activity Log
Resource Monitoring	CloudWatch	Azure Monitor
Threat Detection	GuardDuty / Sec Hub	Defender for Cloud
Secrets & Encryption	KMS / Secrets Mgr	Azure Key Vault
Network Filtering	Security Groups / NACLs	NSGs / Azure Firewall

THREE CORE INFRA PROTECTIONS (BANK SCENARIO)

- **1. Network Segmentation & Private Subnets:** Place DB (RDS) in private subnet with no public IP. Security groups allow port 3306 *only* from API tier. WAF on public ALB/Gateway.
- **2. IAM Least Privilege & MFA:** Stop root account usage; enforce MFA for all users; issue short-lived STS/SSO role credentials instead of long-lived access keys.
- **3. Data Protection:** S3 Block Public Access; encrypt S3 and RDS at rest using **AWS KMS (AES-256)**; TLS 1.3 in transit.
- **IaC Security (Shift-Left Infra):** Write infra in Terraform/CloudFormation; scan in pipeline using **Checkov/tfsec/KICS** before provisioning.

RESILIENCE & ACCESS MODELS

- **RTO:** Max acceptable downtime duration before restore.
- **RPO:** Max acceptable data loss time window (age of data).
- **IDS vs IPS:** *IDS* = passive detection/alert; *IPS* = active inline blocking. **Signature** = known attacks; **Anomaly** = baseline deviation (catches zero-days, more false positives).
- **IAM Models: RBAC** (roles), **ABAC** (attributes: user, time, device), **DAC** (owner decides), **MAC** (strict labels/classifications).

✍ INTENTIONAL BLANK SPACE FOR HANDWRITTEN EXAM NOTES, FORMULAS & CUSTOM DIAGRAMS (SIDE A)

PRESERVED WRITING SPACE

6. Docker Architecture & Container Security

LEC 05 | Q1, Q2(iii), Q4(b)

Docker: Open platform to build, ship, and run applications using OS-level virtualization. **2 Advantages:** 1. Portability/consistency ("runs anywhere"); 2. Lightweight & rapid startup (shares host kernel).

DOCKER ARCHITECTURE COMPONENTS

Client (CLI) → Daemon (dockerd) → Image → Container ↔ Registry

- **Docker Client:** Primary CLI interface (`docker build`, `run`, `pull`).
- **Docker Daemon (`dockerd`):** Background engine managing images, containers, networks, and storage volumes.
- **Docker Image:** Read-only, layered template with app code & deps.
- **Docker Container:** Runnable, isolated instance of an image with a thin writeable top layer.
- **Docker Registry:** Central repository storing & distributing images (Docker Hub, AWS ECR, Harbor).

IMAGE VS CONTAINER & VM COMPARISON

Docker Image	Docker Container
Static, read-only blueprint	Dynamic, running instance
Immutable set of layers	Contains state + writeable layer

VM vs Container: Containers share host OS kernel via cgroups/namespaces (fast, lightweight, but risk of *container escape*). VMs have full guest OS via hypervisor (heavy, stronger hardware isolation).

4 CONTAINER SECURITY RISKS & CONTROLS

- **1. Running as Root:** Compromise gives root on host. → *Add `USER nonroot` (UID 10001); drop caps (`--cap-drop=ALL`).*
- **2. Privileged Container / Socket:** `--privileged` or mounting `/var/run/docker.sock` allows escape. → *Disable privileged mode; never mount docker.sock.*
- **3. Secrets in Image:** Hardcoded `ENV`/`ARG` secrets stored in layers. → *Inject at runtime via Vault/K8s Secrets; use `.dockerignore`.*
- **4. Bloated / Vulnerable Images:** Unpinned tags (`:latest`) contain CVEs. → *Use pinned minimal base (`python:3.12-slim`), scan with Trivy/Grype.*

SECURE VS INSECURE DOCKERFILE

```
# INSECURE DOCKERFILE:
FROM python:latest
ENV DB_PASS=Secret123 \
    API_KEY=key-9876
RUN apt-get update && apt-get install -y gcc vim openssh-se
rver
USER root
EXPOSE 22 5000
```

```
# SECURE DOCKERFILE:
FROM python:3.12-slim
RUN useradd --create-home --uid 10001 appuser
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY --chown=appuser:appuser src/ ./src/
USER appuser
EXPOSE 5000
```

7. Kubernetes (K8s) Architecture & RBAC

LEC 05 | Q1, Q4(c)

K8s Architecture: Control Plane (API Server [mTLS/auth], etcd [encrypted secrets], Scheduler, Controller Mgr). **Worker Nodes** (kubelet, kube-proxy, container runtime, Pods).

KUBERNETES RBAC MODEL & EXAM EXAMPLES

RBAC binds **Subjects** (Users, Groups, ServiceAccounts) to **Roles** (API groups, resources, verbs: `get`, `list`, `create`, `update`, `delete`) via **RoleBindings**.

Entity	Scope & Permissions	Exam Example Configuration
Developer	Namespaced `Role` (least priv)	Allow `get, list, create` on `pods, deploy` in `dev` ns only. **No secret access, no cluster-admin!**
Admin	`ClusterRole` across all ns	`cluster-admin` bound via `ClusterRoleBinding` strictly to dedicated admins with MFA.
ServiceAcct	Pod identity token	1 per app; disable auto-mount (`automountServiceAccountToken: false`).

- **NetworkPolicy:** Default-deny ingress & egress; permit only Frontend → Backend → Database; block egress to internet to prevent exfiltration.

8. DevSecOps CI/CD Pipeline & Tools

LEC 06 | Q1, Q2(ii), Q4(a)

CI/CD Purpose: Automate building, testing, validating, packaging, and deploying software. Integrates security checks to block bad code.

Continuous Delivery = deployable build + manual release gate. **Continuous Deployment** = automatic release to production without manual intervention.

6-STAGE DEVSECOPS PIPELINE & TOOL MAPPING

Pipeline Stage	Security Check / Activity	Example Tools
1. Code / Commit	Pre-commit hooks, **Secret scanning**, branch rules	**Gitleaks**, TruffleHog, GitHub Secret Scan
2. Build	**SAST** (source code) + **SCA** (dependencies)	**SonarQube**, Semgrep, **Snyk**, Dependabot
3. Package	Container image scanning, **SBOM**, artifact sign	**Trivy**, Grype, CycloneDX, **Cosign**
4. Test / Staging	**DAST** (runtime attack) + **IaC scanning**	**OWASP ZAP**, **Checkov**, tfsec, KICS
5. Deploy	**Security Gate**, admission control policies	OPA Gatekeeper, Kyverno
6. Runtime	Continuous monitoring, runtime threat detection	Falco, Wazuh, SIEM (Sentinel, Splunk)

SECURITY TESTING MATRIX & SECURITY GATES

Type	Analyzes	Mode	Key Detection
SAST	Source code / AST	White-box (no run)	SQLi patterns, hardcoded secrets
SCA	3rd-party libs	Dependencies	Known CVEs, outdated packages
DAST	Running HTTP API	Black-box (dynamic)	XSS, auth bypass, misconfigs
IaC	Terraform / K8s	Config files	Public S3, open SGs, root pods

Security Gate: Automated pipeline policy condition that stops build and blocks deployment if failed (e.g. *0 Critical CVEs, 0 exposed secrets, signed image*). Leaked secrets must be **revoked & rotated immediately** (deleting commit is not enough).

9. Continuous Monitoring & SIEM

LEC 07 | Q1, Q4(e)

Continuous Security Monitoring (CSM): Real-time observability of infrastructure, container, and cloud telemetry. Feedback loops back to SDLC Plan.

- **Why Important:** Early threat detection, rapid containment, visibility into cloud actions, compliance & audit readiness.
- **Log Management:** Sync clocks via NTP (UTC). Mandatory fields: timestamp, src IP, user ID, action, result. Tamper-proof WORM storage. Redact PII.
- **SIEM (Security Information & Event Management):** Centralizes, normalizes, and correlates logs from multiple sources (CloudTrail, K8s audit, firewalls). Tools: **Microsoft Sentinel, Splunk, Elastic**.
- **Key Triggers:** Unusual outbound data transfer (exfiltration alerts), multiple failed logins, privileged pod creation, bulk secret reads.
- **Alert Accuracy:** *False Positive* = benign flagged bad; *False Negative* = breach missed (dangerous!).

10. Incident Response (IR) 6-Phase Lifecycle

LEC 07 | Q1, Q4(f)

1. Prep → 2. Detect → 3. Contain → 4. Eradicate → 5. Recover → 6. Lessons

- **1. Preparation:** Establish IR plan, team roles, communication channels, tested backups, and access to forensic tools.
- **2. Detection & Analysis:** Validate alerts, scope affected systems/data, determine attack vector, analyze indicators of compromise (IOCs).
- **3. Containment:** *Short-term:* Isolate compromised pods/nodes, revoke stolen tokens, block attacker IP. *Long-term:* Segment network to prevent lateral spread.
- **4. Eradication:** Remove malware/malicious packages, patch exploited vulnerabilities, re-image nodes, rotate DB passwords & all secrets.
- **5. Recovery:** Restore clean systems from verified backups, test in staging, gradually return to production, heighten monitoring.
- **6. Lessons Learned:** Post-incident review, root-cause analysis (RCA), update pipeline security gates, report breach to regulator within **72 hours (GDPR Art 33)**.
- **Digital Forensics & Chain of Custody:** Work on **bit-stream copies** (never live disk); verify SHA-256 hashes; log every transfer and custodian to maintain legal evidence integrity.

11. Governance, Risk, Compliance & Healthcare

LEC 08 | Q1, Q4(d)

GRC: Governance (oversight/policies) + **Risk** (management) + **Compliance** (legal proof). **Compliance ≠ Security** (compliance is minimum baseline).

Regulation / Body	Type	Core Scope & Exam Requirements
GDPR	EU Statutory Law	Protects personal data. 7 Principles. **Article 33: Notify regulator within 72h**. Extraterritorial.
HIPAA	US Statutory Law	Protects **PHI / ePHI**. Privacy Rule, Security Rule, Breach Notification Rule.
ENISA	EU Agency	**Advisory Agency (NOT law!)**. Produces guidelines & threat reports.

3 HEALTHCARE CONTROLS TO PROTECT PATIENT DATA

- **1. Technical:** Encrypt at rest (AES-256) & transit (TLS 1.3); MFA; RBAC.
- **2. Administrative:** Access review policies, staff security training, IR plan.
- **3. Physical:** Data center access badges, encrypted workstation disks.
- **Control Classes:** *Preventive* (MFA, WAF), *Detective* (SIEM, IDS), *Corrective* (backups, patch), *Compensating* (isolated VLAN if no MFA).

12. Secure Serverless Architectures & DoW

LEC 10 | Q1

Serverless / FaaS (Lambda): Event-driven, auto-scaling, ephemeral execution. *CSP manages hardware, OS, runtime; Customer manages code & IAM!*

- **Denial of Wallet (DoW):** Malicious flood of event triggers exhausts cloud budget instead of crashing system. → *API rate limiting/throttling, concurrency limits, budget kill-switches & billing alarms.*
- **Serverless IAM:** Scoped per-function least privilege (limits blast radius; never share single AdministratorAccess role).

✍ INTENTIONAL BLANK SPACE FOR HANDWRITTEN EXAM NOTES, FORMULAS & CUSTOM DIAGRAMS (SIDE B) — PRESERVED WRITING SPACE